CircuitBreak — Team Step-by-Step Plan

Topic (fixed): CircuitBreak: Cooperative Networked Electronics Escape Room **Deadline:** Sat 14 Nov 2026 · **Plan written:** Tue 20 Oct 2026 **This file replaces** `docs/PROJECT_OVERVIEW.md`, `docs/members/*` and `plans/*`, which describe an older puzzle design.

Part 1 — The game in plain words

Three players are locked in a lab. The only exit is an **electronic door controller**, and it is broken. They repair it together to escape.

Each player sits at a different laptop and sees only one part of the problem:

Role	Has	Can do
Inspector	The repair manual	Read the manual, press ARM
Analyst	The measuring instruments	Measure parts, read the lamp
Operator	The tools and controls	Replace, flip and set parts, press ARM

Nobody can finish alone, so they have to talk.

The door has two stages. Both must be fixed:

1. **Power stage.** The door has no power. The server secretly breaks **one** of three things at random:
   - the fuse is blown;
   - the battery is in backwards;
   - the diode is in backwards.

   The Analyst measures, the Inspector compares against the manual, and the Operator fixes the right part.
2. **Lock stage.** The lock opens only when a logic gate outputs 1. The Operator sets three switches, the Analyst watches the lamp, and the Inspector has the gate diagram. When the lamp is on, the Inspector and Operator must both type `arm` within 1.5 seconds of each other.

Fix both stages and the door opens: **ESCAPED**. If the timer runs out first: **FAILED**.

Two more stages (a 7-segment display and a Morse code LED) are **optional**. Add them only after everything above works.

Part 2 — The four Java ideas you need (and nothing else)

Learn these four. The whole project is built from them.

Idea	Everyday picture	Java you write
Socket	A phone call between two programs	Server: `new ServerSocket(6600)` then `accept()`. Client: `new Socket(ip, 6600)`

Idea	Everyday picture	Java you write
Sending and receiving lines	Talking and listening on the call	`out.println("hello")` to send; `in.readLine()` to receive (it **waits** until a line arrives)
Thread	Hiring an extra worker so two jobs happen at once	`new Thread(() -> { ... }).start();`
Lock (`synchronized`)	One marker for a shared whiteboard: only the person holding it can write	`synchronized void doThing() { ... }`

Why the game needs each one:

- **Socket.** Three laptops must talk to one server.
- **Thread.** The server has to listen to all three players at the same time. Each client has to listen to the server while its player is typing.
- **Lock.** Two players can act at exactly the same moment. Only one change to the door may happen at a time.

You already have working examples of all four:

- the lecturer's workshop code: `Chat_Thread.zip` and `basic thread program.zip`;
- our `circuitbreakserver.java`.

Part 3 – Who does what

Each person owns **one** piece and can build and test it **alone**. The pieces join up in week 2.

	Member 1 – Server	Member 2 – Client	Member 3 – Door logic
Builds	The program the three laptops connect to	The program each player runs	The rules of the broken door
Java used	Sockets, threads, `synchronized`	Socket, one extra thread, `Scanner`	Classes, `if`/`else`, `Random`, strings
Difficulty	Hardest	Easy	Easiest (no networking at all)
Files	`CircuitBreakServer.java`, `GameRoom.java`	`CircuitBreakClient.java`	`Stage.java`, `PowerStage.java`, `LockStage.java`, `StageTest.java`
Lines of code (approx.)	150	60	150
LO evidence	LO-5 + LO-6	LO-5 + LO-6	LO-6 (lock test)

Put the strongest Java person on Member 1. The weakest can start as Member 3, because it is plain Java with no networking.

The contract between the pieces (agree on this first, day 1)

Every message is **one line of plain text**.

What players type (the client sends it exactly as typed):

```
join inspector | join analyst | join operator
ready
manual                          (Inspector)
measure fuse|battery|diode      (Analyst)
lamp                            (Analyst)
replace fuse                    (Operator)
flip battery | flip diode       (Operator)
set 101                         (Operator: switches A, B, C)
arm                             (Inspector or Operator)
quit
```

What the server sends back: plain lines such as:

```
WELCOME analyst
FUSE: no continuity (OPEN)
ERROR: only the Operator can do that
STAGE 2: LOCK
DOOR OPEN - ESCAPED
```

Member 3's classes, exactly:

```java
public enum Role { INSPECTOR, ANALYST, OPERATOR }

public interface Stage {
    String name();                       // "POWER" or "LOCK"
    String handle(Role who, String cmd); // returns the reply text for that player
    boolean isSolved();
}
```

Member 1's server only ever calls `handle(...)` and `isSolved()`. Member 3 never touches sockets. That separation is what lets three beginners work in parallel.

Part 4 — Week by week

Week 0 — Learn by running (Tue 20 – Fri 23 Oct)

Everyone does all four steps. Nobody writes project code yet.

1. Install JDK 21. Check it with `javac -version`.
2. Unzip `Chat_Thread.zip`. Run `ChatServerT` in one terminal and `ChatClientT` in two others. Chat between them.
3. Run our `circuitbreakserver.java` with three `nc localhost 6600` terminals (or three copies of the client). Close one and show the other two still work.
4. Each person explains, out loud and in their own words, to the other two:
    - What does `accept()` do?
    - What does `readLine()` wait for?
    - Why does the server need a thread per client?

Done when: all three have run the code on their own laptop and answered the three questions. Fix this before anything else; it is the foundation for everything.

Week 1 — Build your piece alone (Sat 24 – Fri 30 Oct)

Member 3 — door logic

1. Write `Role` and `Stage` exactly as above.
2. Write `PowerStage`:
    - In the constructor, use `new Random()` to pick one fault: `FUSE`, `BATTERY` or `DIODE`.
    - `handle(Role, cmd)` answers each command for each role:

Command	Who	Healthy part	Broken part
`manual`	Inspector	Prints the rules on the right	–
`measure fuse`	Analyst	`OK (continuity)`	`OPEN (no continuity)`
`measure battery`	Analyst	`+9.0 V`	`-9.0 V`
`measure diode`	Analyst	`0.7 V forward`	`no conduction`
`replace fuse` / `flip battery` / `flip diode`	Operator	Wrong part: reply `nothing changed - 30 s penalty`	Right part: fixed, `isSolved()` becomes `true`

    The manual text: “Fuse must show continuity. Battery must read +9 V. Diode must conduct at 0.7 V.”

    Any other role trying an Operator command gets `ERROR: only the Operator can do that`.
3. Write `LockStage`:
    - Switches start at `000`. `set 101` sets them.
    - Lamp = `(A XOR B) AND C`.
    - `arm`: record the time with `System.nanoTime()` and which role pressed it. If both the Inspector and the Operator have armed within 1.5 s **while the lamp is on**, the stage is solved.
    - Make `handle` `synchronized`, because two players can press `arm` at once.
4. Write `StageTest` with a `main` method:
    - Try every command for every role and print the replies.
    - Then start two threads that both call `arm` at the same moment, 1000 times. Check the stage is solved exactly once each run.

Done when: `java StageTest` prints correct replies for all three faults, and the two-thread test passes.

Member 2 – client

1. `CircuitBreakClient <server-ip>` connects to port 6600.
2. Start **one extra thread** that loops on `in.readLine()` and prints every line it receives.
3. The main thread loops on `Scanner.nextLine()` and sends each typed line with `out.println(...)`.
4. When `readLine()` returns `null`, print “Server closed the connection” and exit.
5. Test against our existing echo server: type a line and see it echoed back. Then have a second laptop’s message appear **without pressing Enter**.

Done when: the client works against `circuitbreakserver.java` from a second laptop, and a wrong IP prints a friendly error.

Member 1 – server

1. Start from `circuitbreakserver.java`. Keep one thread per client.
2. The first line from a client must be `join <role>`. Store the role for that connection. Refuse a role that is already taken, and refuse a fourth player.

3. Write `GameRoom` with `synchronized` methods. It holds the current stage (`PowerStage`, then `LockStage`) and a reference to the other players' output streams.
4. For every other line: `reply = stage.handle(role, line)`. Send the reply to that player. Send important events (`STAGE 2`, `ESCAPED`) to all three.
5. After every command, if `stage.isSolved()`, move to the next stage.
6. Until Member 3's classes exist, use a fake `Stage` that answers `"ok: " + cmd`.

Done when: three `nc` terminals can join with different roles, a duplicate role is refused, and commands reach the fake stage.

Week 2 — Join the pieces (Sat 31 Oct – Fri 6 Nov)

1. **Sat 31 Oct, everyone together, about 2 hours:** copy all files into `src/` and compile with `javac -d out src/*.java`. Replace the fake stage with Member 3's classes. Play one full game on one laptop with three terminals.
2. Then play on **three laptops** on the same Wi-Fi. Fix firewall and IP problems now, not on demo day.
3. Add the countdown: a timer thread in the server calls `GameRoom.tick()` every second. Send everyone the time each minute and every second for the last 10 seconds. At 0, send `FAILED`.
4. Run the tests in Part 5 and write down the actual results.

Done when: a full game reaches ESCAPED on three laptops, and a game left alone reaches FAILED.

Week 3 — Polish, evidence, rehearsal (Sat 7 – Fri 13 Nov)

- **Optional:** add the display or Morse stage, but only if week 2 is completely done.
- Write the report: game description, how sockets and threads are used, test results table, known limits.
- Each member explains their own code and traces one command end to end: Operator types `replace fuse` → client → server thread → `GameRoom` → `PowerStage` → replies on all three screens.
- Rehearse the demo twice. Record one backup video.

Sat 14 Nov: submit.

Part 5 — Tests to run and record

Record each test in a table: date, test, expected result, actual result, who ran it.

1. Three players join with three different roles. A fourth is refused.
2. Two players ask for the same role. The second is refused.
3. The Inspector types `replace fuse`. It is refused, and the door does not change.
4. For each of the three faults: the Analyst's measurements identify it, and the right fix solves the stage.
5. Wrong fix → `30 s penalty`, and the stage stays unsolved.
6. Lamp off + `arm` → refused. Arms 3 s apart → not solved. Arms within 1.5 s → `ESCAPED`.
7. Two threads `arm` at once, 1000 runs → solved exactly once every run.
8. Close one client mid-game. The other two stay connected.
9. The timer reaches 0 → `FAILED`, sent once.
10. A full game on three separate laptops.

Part 6 — How we work together

- **15-minute call every Tue, Thu and Sat.** Each person shows their code running, says what blocks them, and agrees on their next small step.

- **Stuck for more than 30 minutes?** Message the group with the exact error text. Pair with someone for 20 minutes.
- **Done means:** it runs on your laptop, it's pushed to the shared repo, and you can explain it without notes.
- **AI tools:** fine for learning, but you must be able to explain every line you submit. The viva will test this.

Part 7 — What each person must be able to answer in the viva

Everyone

- What is a socket?
- Why TCP?
- What does `readLine()` wait for?
- Why do we need threads?
- What does `synchronized` protect?

Member 1

- Difference between `ServerSocket` and `Socket`.
- Why one thread per client.
- How the server knows which role sent a command.

Member 2

- Why the client needs a second thread.
- What happens when the server closes.
- Why `localhost` doesn't work across laptops.

Member 3

- How each fault shows up in the measurements.
- The truth table of `(A XOR B) AND C`.
- What goes wrong without `synchronized` when two players arm together.